

1. In Java, se l'unico polimorfismo universale usato è quello per inclusione, allora tutti gli eventuali errori di tipo sono segnalati a tempo di compilazione NO
2. Il primo Fortran supportava un analogo della malloc NO
3. Il Prolog puro supporta i cicli for/next NO
4. Java non ha metodi analoghi alla funzione free() di C e C++ SI
5. Se in Java si usa unicamente il polimorfismo parametrico allora tutti controlli di tipo avvengono a tempo di compilazione. SI
6. Nel paradigma funzionale puro non ci sono gli assegnamenti. SI
7. Nel primo Fortran l'occupazione di memoria di un programma era nota a compile time. SI
8. HTTP (senza scripts) è un linguaggio general purpose. NO
9. Il C++ ha un garbage collector NO
10. La JVM può caricare classi da host diversi SI
11. Un linguaggio puramente funzionale ha i cicli while NO
12. La promozione automatica dei tipi numerici è una forma di polimorfismo ad hoc SI
13. Il C adotta sempre la structural equivalence tra tipi NO
14. Il predicato Prolog member può enumerare i membri di una lista e generare liste SI
15. Tra i compiti del supporto a run time c'è la gestione della memoria SI
16. I linguaggi funzionali non sono computazionalmente completi NO
17. In C++ l'ereditarietà multipla può causare un consumo esponenziale di memoria SI
18. Il primo Fortran supportava la ricorsione. NO
19. Se il linguaggio è dinamicamente tipato, allora il tipo di una variabile può cambiare durante l'esecuzione del programma. SI
20. C adotta la structural equivalence per le struct e la name equivalence per gli altri tipi. NO
21. Si può accedere alle variabili non locali di una procedura in tempo costante, indipendentemente da quanti record di attivazione si devono attraversare. SI
22. I linguaggi funzionali puri non sono computazionalmente completi
NO
23. Compilatore e programmi oggetto sono di norma eseguiti dalla stessa macchina NO
24. C è debolmente tipato SI

25. Nei linguaggi fortemente tipati il type checking avviene tutto a tempo di compilazione NO

SPECIALE

Dire se il polimorfismo parametrico puro (quello dei template):

1. E' più adatto del polimorfismo per inclusione per rappresentare insiemi di oggetti omogenei SI [X] NO []
2. E' più adatto del polimorfismo per inclusione per rappresentare insiemi di oggetti eterogenei SI [] NO [X]
3. Ammette il controllo di tipi completo a tempo di compilazione SI [X] NO []
4. Può generare errori di tipo a runtime SI [] NO [X]